Структуры данных и методы их обработки

Коллекции в языке Python

Содержание лекции

1. Понятие коллекции. Классификация
2. Последовательности: списки и кортежи
3. Словари и пары ключ–значение
4. Множества и операции над ними
5. Генераторы коллекций
6. Сортировка и упорядочивание
7. Типичные схемы обработки данных
8. Модуль `collections`
9. Итоги и вопросы

1. Понятие коллекции

Коллекция — структурированное хранилище, содержащее множество элементов и предоставляющее операции для добавления, удаления, поиска и перебора.

В Python все коллекции:

- являются **итерируемыми объектами** (поддерживают протокол `for ... in`);
- реализуют определённые методы доступа;
- различаются по семантике хранения элементов.

Классификация коллекций

Критерий	Типы
Упорядоченность	`list`, `tuple`, `dict` / `set`, `frozenset`
Изменяемость	`list`, `dict`, `set` / `tuple`, `frozenset`
Доступ	По индексу / По ключу / По значению
Дубликаты	Допускаются (`list`, `tuple`) / Не допускаются (`set`)

2. Последовательности: списки

Список (`list`) — изменяемая упорядоченная последовательность.

Реализация: динамический массив указателей.

```
primes = [2, 3, 5, 7, 11]
mixed = [42, "hello", 3.14, None]
generated = list(range(10))
```

- Доступ по индексу: **O(1)**
- Добавление в конец: **O(1)** амортизированно
- Вставка в начало: **O(n)**

Операции со списками

```
a = [10, 20, 30, 40, 50]

a[0]           # 10 — доступ по индексу
a[1:4]         # [20, 30, 40] — срез
a[::-1]        # [50, 40, 30, 20, 10] — реверс
a.append(60)   # добавление в конец
a.insert(0, 5)  # вставка по индексу
a.pop()        # удаление последнего
a.remove(30)   # удаление по значению
```

Сложность операций со списком

Операция	Сложность
`a[i]`	O(1)
`a.append(x)`	O(1)*
`a.insert(0, x)`	O(n)
`a.pop()`	O(1)
`x in a`	O(n)
`a.sort()`	O(n log n)
`a + b`	O(n + m)

* — *амортизированная сложность*

Кортежи (`tuple`)

Кортеж — неизменяемая упорядоченная последовательность.

```
point = (3.0, 4.0)
singleton = (42,)          # запятая обязательна!
x, y = point               # распаковка
```

Применение:

- Возврат нескольких значений из функции
- Ключи словарей (кортежи хешируемы)
- Неизменяемые записи данных

3. Словари (`dict`)

Словарь — коллекция пар «ключ–значение» с доступом по ключу.

Реализация: **хеш-таблица** с открытой адресацией.

С Python 3.7 гарантируется **порядок вставки**.

```
student = {
    "name": "Иванов",
    "age": 20,
    "grades": [5, 4, 5, 3]
}
student["name"]              # "Иванов"
student.get("email", "N/A")  # безопасный доступ
```

Операции со словарями

```
d = {"a": 1, "b": 2, "c": 3}

d.keys()       # dict_keys(['a', 'b', 'c'])
d.values()     # dict_values([1, 2, 3])
d.items()      # dict_items([('a',1), ('b',2), ('c',3)])

d.update({"c": 30, "d": 4})  # слияние
del d["a"]                    # удаление
v = d.pop("b")               # удаление с возвратом

# Слияние (Python 3.9+)
merged = d | {"e": 5}
```

Требования к ключам словаря

Ключ должен быть **хешируемым**:

- Реализует `__hash__()` — инвариантен за время жизни
- Реализует `__eq__()`
- Если `a == b`, то `hash(a) == hash(b)`

Хешируемые ✅	Нехешируемые ❌
`int`, `float`, `str`	`list`
`tuple` (если элементы хешируемы)	`dict`
`frozenset`, `bytes`	`set`

4. Множества (`set`, `frozenset`)

Множество — неупорядоченная коллекция уникальных хешируемых элементов.

Реализация: хеш-таблица (как словарь без значений).

```
colors = {"red", "green", "blue"}
numbers = set([1, 2, 2, 3, 3])  # {1, 2, 3}
empty_set = set()                # НЕ {}!
```

Проверка принадлежности: **O(1)** в среднем.

Операции над множествами

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

A | B    # объединение:    {1,2,3,4,5,6,7,8}
A & B    # пересечение:    {4, 5}
A - B    # разность:       {1, 2, 3}
A ^ B    # симм. разность: {1, 2, 3, 6, 7, 8}
A <= B   # подмножество:   False
```

`frozenset` — неизменяемый аналог, может быть ключом словаря.

5. Генераторы коллекций

Списковое включение (list comprehension):

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(50) if x % 2 == 0]
matrix = [[i*j for j in range(5)] for i in range(5)]
```

Словарное и множественное включения:

```
char_count = {ch: text.count(ch) for ch in set(text)}
unique = {x % 5 for x in range(20)}
```

Генераторное выражение (ленивое вычисление):

```
gen = (x**2 for x in range(1000000))  # не хранит всё в памяти
```

6. Сортировка и упорядочивание

```
data = [3, 1, 4, 1, 5, 9, 2, 6]

sorted(data)                    # новый список
sorted(data, reverse=True)      # по убыванию
sorted(data, key=lambda x: -x)  # по ключу

# Метод списка — сортировка на месте
data.sort()
data.sort(key=str)
```

- Алгоритм: **Timsort** — O(n log n)
- Сортировка **устойчивая**: равные элементы сохраняют относительный порядок

7. Типичные схемы обработки данных

Фильтрация

```
adults = [p for p in people if p["age"] >= 18]
```

Агрегирование

```
total = sum(item["price"] * item["qty"] for item in cart)
```

7. Типичные схемы обработки данных

Подсчёт

```
count = sum(1 for x in data if x > 0)
```

Группировка

```
from collections import defaultdict
groups = defaultdict(list)
for item in data:
    groups[item["category"]].append(item)
```

Преобразование данных

```
# map — применение функции к каждому элементу
squared = list(map(lambda x: x**2, numbers))

# filter — отбор по предикату
odds = list(filter(lambda x: x % 2 != 0, numbers))

# zip — параллельная итерация
names = ["Алиса", "Борис", "Вера"]
scores = [95, 87, 91]
pairs = list(zip(names, scores))

# reduce — свёртка
from functools import reduce
product = reduce(lambda a, b: a * b, [1, 2, 3, 4, 5])
```

8. Модуль `collections`

Тип	Назначение
`namedtuple`	Кортеж с именованными полями
`deque`	Двусторонняя очередь, O(1) на обоих концах
`Counter`	Мультимножество, подсчёт частот
`defaultdict`	Словарь со значением по умолчанию
`OrderedDict`	Словарь с `move_to_end()`
`ChainMap`	Объединение нескольких словарей

`Counter` — подсчёт частот

```python
from collections import Counter

text = "абракадабра"
freq = Counter(text)
# Counter({'а': 5, 'б': 2, 'р': 2, 'к': 1, 'д': 1})

freq.most_common(2)  # [('а', 5), ('б', 2)]

a = Counter(x=1, y=2)
b = Counter(y=3, z=1)
a + b   # Counter({'y': 5, 'x': 1, 'z': 1})
```

`deque` — двусторонняя очередь

```
from collections import deque

dq = deque([1, 2, 3], maxlen=5)
dq.appendleft(0)   # O(1) — слева
dq.append(4)       # O(1) — справа
dq.popleft()       # O(1) — извлечение слева
```

> В отличие от `list`, где `insert(0, x)` и `pop(0)` требуют O(n), `deque` реализован как двусвязный список блоков.

`defaultdict` и `namedtuple`

```python
from collections import defaultdict, namedtuple

# defaultdict — граф смежности
graph = defaultdict(list)
for u, v in [(1,2), (1,3), (2,4)]:
    graph[u].append(v)

# namedtuple — запись с полями
Point = namedtuple("Point", ["x", "y"])
p = Point(3.0, 4.5)
print(p.x, p.y)  # 3.0 4.5
```

Выбор структуры данных

Задача	Структура
Упорядоченная последовательность	`list`
Неизменяемая запись / ключ	`tuple`
Проверка принадлежности	`set`
Доступ по ключу	`dict`
Очередь / стек	`deque`
Подсчёт частот	`Counter`
Группировка	`defaultdict(list)`

Типичная ошибка производительности

```
# ❌ ПЛОХО: O(n²)
result = []
for x in large_list:
    if x not in result:   # O(n) для list!
        result.append(x)

# ✅ ХОРОШО: O(n)
seen = set()
result = []
for x in large_list:
    if x not in seen:     # O(1) для set
        seen.add(x)
        result.append(x)
```

Итоги лекции

- Коллекции — фундамент работы с данными в Python
- Выбор структуры данных определяет эффективность алгоритма
- Генераторы коллекций обеспечивают декларативный стиль обработки
- Модуль `collections` расширяет стандартные возможности
- Понимание хешируемости критично для работы со словарями и множествами

Вопросы для самопроверки

1. Чем список отличается от кортежа по семантике и производительности?
2. Почему список не может быть ключом словаря?
3. Какова сложность проверки `x in set` и почему?
4. Чем генераторное выражение отличается от спискового включения?
5. В каких случаях следует использовать `deque` вместо `list`?
6. Что такое хешируемость и зачем она нужна?

Задача 1. Анализ результатов экзамена

Условие задачи

Имеется список баллов студентов за экзамен (от 0 до 100):

```
scores = [78, 92, 55, 83, 64, 91, 72, 88, 60, 45, 97, 81, 39, 76, 85]
```

Требуется:

1. Найти средний балл.
2. Определить количество сдавших (балл ≥ 60).
3. Отсортировать по убыванию.
4. Сформировать список «отличников» (балл ≥ 90).

Шаг 1. Средний балл

```
scores = [78, 92, 55, 83, 64, 91, 72, 88, 60, 45, 97, 81, 39, 76, 85]

average = sum(scores) / len(scores)
print(f"Средний балл: {average:.2f}")
# Средний балл: 73.07
```

> **Комментарий:** `sum()` и `len()` — встроенные функции, работающие с любой последовательностью за O(n). Формат `:.2f` ограничивает вывод двумя знаками после запятой.

Шаг 2. Подсчёт сдавших

```
passed = sum(1 for s in scores if s >= 60)
print(f"Сдали: {passed} из {len(scores)}")
# Сдали: 13 из 15
```

Альтернативный вариант через фильтрацию:

```
passed_list = [s for s in scores if s >= 60]
print(f"Сдали: {len(passed_list)}")
```

> Здесь используется **генераторное выражение** внутри `sum()` — экономия памяти по сравнению с созданием списка.

Шаг 3. Сортировка по убыванию

```
sorted_desc = sorted(scores, reverse=True)
print(sorted_desc)
# [97, 92, 91, 88, 85, 83, 81, 78, 76, 72, 64, 60, 55, 45, 39]
```

> **Важно:** `sorted()` возвращает **новый** список, не изменяя исходный.
>
> Метод `scores.sort(reverse=True)` изменяет список на месте.

Шаг 4. Список отличников

```
excellent = [s for s in scores if s >= 90]
print(f"Отличники: {excellent}")
# Отличники: [92, 91, 97]

# Если нужна сортировка по убыванию:
excellent = sorted([s for s in scores if s >= 90], reverse=True)
print(f"Отличники: {excellent}")
# Отличники: [97, 92, 91]
```

Итоговый код задачи 1

```
scores = [78, 92, 55, 83, 64, 91, 72, 88, 60, 45, 97, 81, 39, 76, 85]

average   = sum(scores) / len(scores)
passed    = sum(1 for s in scores if s >= 60)
top       = sorted(scores, reverse=True)
excellent = sorted([s for s in scores if s >= 90], reverse=True)

print(f"Средний балл: {average:.2f}")
print(f"Сдали: {passed}/{len(scores)}")
print(f"Рейтинг: {top[:5]} ...")
print(f"Отличники: {excellent}")
```

Задача 2. Частотный анализ текста

Условие задачи

Дан текст. Требуется:

1. Подсчитать частоту каждого слова.
2. Вывести 5 самых частых слов.
3. Определить количество уникальных слов.
4. Найти слова, встречающиеся ровно 1 раз.

```
text = """
Программирование есть искусство разбиения больших задач
на маленькие. Программирование требует терпения и логики.
Искусство программирования приходит с практикой.
"""
```

Шаг 1. Токенизация и нормализация

```
# Разбиваем текст на слова, убираем знаки препинания
words = []
for token in text.split():
    word = token.strip(".,!?;:").lower()
    if word.isalpha():
        words.append(word)

print(words[:6])
# ['программирование', 'есть', 'искусство', 'разбиения',
#  'больших', 'задач']
```

> **Комментарий: `split()` разбивает по пробельным символам. `strip()` удаляет указанные символы с обоих концов строки. `isalpha()` отбрасывает «пустые» токены.

Шаг 2. Подсчёт частот через словарь

```
freq = {}
for word in words:
    if word in freq:
        freq[word] += 1
    else:
        freq[word] = 1

print(freq)
# {'программирование': 2, 'есть': 1, 'искусство': 2, ...}
```

Более идиоматичный вариант:

```
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1
```

Шаг 3. То же через `Counter`

```
from collections import Counter

freq = Counter(words)
print(freq.most_common(5))
# [('программирование', 2), ('искусство', 2),
#  ('есть', 1), ('разбиения', 1), ('больших', 1)]
```

> `Counter` — подкласс `dict`, специально созданный для подсчёта хешируемых объектов. `most_common(n)` возвращает *n* самых частых элементов.

Шаг 4. Уникальные и редкие слова

```
# Количество уникальных слов
unique_count = len(freq)
print(f"Уникальных слов: {unique_count}")

# Слова, встречающиеся ровно 1 раз
hapax = [word for word, count in freq.items() if count == 1]
print(f"Встречаются 1 раз: {hapax}")
# Встречаются 1 раз: ['есть', 'разбиения', 'больших',
#   'задач', 'на', 'маленькие', 'требует', 'терпения', ...]
```

Итоговый код задачи 2

```
from collections import Counter

text = """..."""  # исходный текст

words = [
    token.strip(".,!?;:").lower()
    for token in text.split()
    if token.strip(".,!?;:").isalpha()
]

freq = Counter(words)

print(f"Всего слов: {len(words)}")
print(f"Уникальных: {len(freq)}")
print(f"Топ-5: {freq.most_common(5)}")
hapax = [w for w, c in freq.items() if c == 1]
print(f"Ровно 1 раз: {len(hapax)} слов")
```

Задача 3. Инвентаризация склада

Условие задачи

Имеется журнал прихода и расхода товаров:

```
transactions = [
    ("болты",      "приход", 500),
    ("гайки",      "приход", 300),
    ("болты",      "расход", 120),
    ("шайбы",      "приход", 1000),
    ("гайки",      "расход", 80),
    ("болты",      "приход", 200),
    ("шайбы",      "расход", 350),
    ("гайки",      "приход", 150),
]
```

Требуется: определить итоговый остаток каждого товара.

Шаг 1. Анализ данных

Каждая запись — кортеж `(товар, операция, количество)`.

- **Приход** увеличивает остаток.
- **Расход** уменьшает остаток.

Шаг 2. Простое решение через `dict`

```
stock = {}

for item, operation, qty in transactions:
    if item not in stock:
        stock[item] = 0

    if operation == "приход":
        stock[item] += qty
    else:
        stock[item] -= qty

print(stock)
# {'болты': 580, 'гайки': 370, 'шайбы': 650}
```

> Работает, но содержит лишнюю проверку `if item not in stock`.

Шаг 3. Улучшение: `defaultdict`

```
from collections import defaultdict

stock = defaultdict(int)  # значение по умолчанию: 0

for item, operation, qty in transactions:
    if operation == "приход":
        stock[item] += qty
    else:
        stock[item] -= qty

print(dict(stock))
# {'болты': 580, 'гайки': 370, 'шайбы': 650}
```

> `defaultdict(int)` автоматически создаёт значение `0` при первом обращении к ключу — проверка `if item not in stock` не нужна.

Шаг 4. Улучшение: `namedtuple` для записей

```
from collections import namedtuple

Transaction = namedtuple("Transaction", ["item", "op", "qty"])

transactions = [
    Transaction("болты",  "приход", 500),
    Transaction("гайки",  "приход", 300),
    Transaction("болты",  "расход", 120),
    # ...
]

# Доступ по именам полей вместо индексов:
for t in transactions:
    print(t.item, t.op, t.qty)
```

> Код становится **самодокументируемым**: `t.item` понятнее, чем `t[0]`.

Шаг 5. Расширение: отчёт по категориям

```
# Добавим группировку: какой товар приходил, какой уходил
from collections import defaultdict

income = defaultdict(int)
expense = defaultdict(int)

for t in transactions:
    if t.op == "приход":
        income[t.item] += t.qty
    else:
        expense[t.item] += t.qty

# Итоговая таблица
for item in sorted(set(income) | set(expense)):
    balance = income[item] - expense[item]
    print(f"  {item:8s}  приход: {income[item]:5d}  "
          f"расход: {expense[item]:5d}  остаток: {balance}")
```

Вывод задачи 3

```
болты     приход:   700  расход:   120  остаток:   580
гайки     приход:   450  расход:    80  остаток:   370
шайбы     приход:  1000  расход:   350  остаток:   650
```

Ключевые приёмы:

- `defaultdict(int)` — устраняет ручную инициализацию.
- `namedtuple` — делает записи читаемыми.
- Разделение на `income` / `expense` — упрощает отчётность.

Итоговый код задачи 3

```
from collections import defaultdict, namedtuple

Transaction = namedtuple("Transaction", ["item", "op", "qty"])

transactions = [
    Transaction("болты",  "приход", 500),
    Transaction("гайки",  "приход", 300),
    Transaction("болты",  "расход", 120),
    Transaction("шайбы",  "приход", 1000),
    Transaction("гайки",  "расход", 80),
    Transaction("болты",  "приход", 200),
    Transaction("шайбы",  "расход", 350),
    Transaction("гайки",  "приход", 150),
]

stock = defaultdict(int)
for t in transactions:
    stock[t.item] += t.qty if t.op == "приход" else -t.qty

for item, qty in sorted(stock.items()):
    print(f"  {item}: {qty}")
```

Сравнение трёх задач

Задача	Структуры	Ключевой приём
1. Экзамен	`list`	Включения, `sorted()`, `sum()`
2. Текст	`dict`, `Counter`	Подсчёт частот, `most_common()`
3. Склад	`defaultdict`, `namedtuple`	Агрегация, группировка

Общий принцип: выбор структуры данных определяется **характером доступа** к данным.